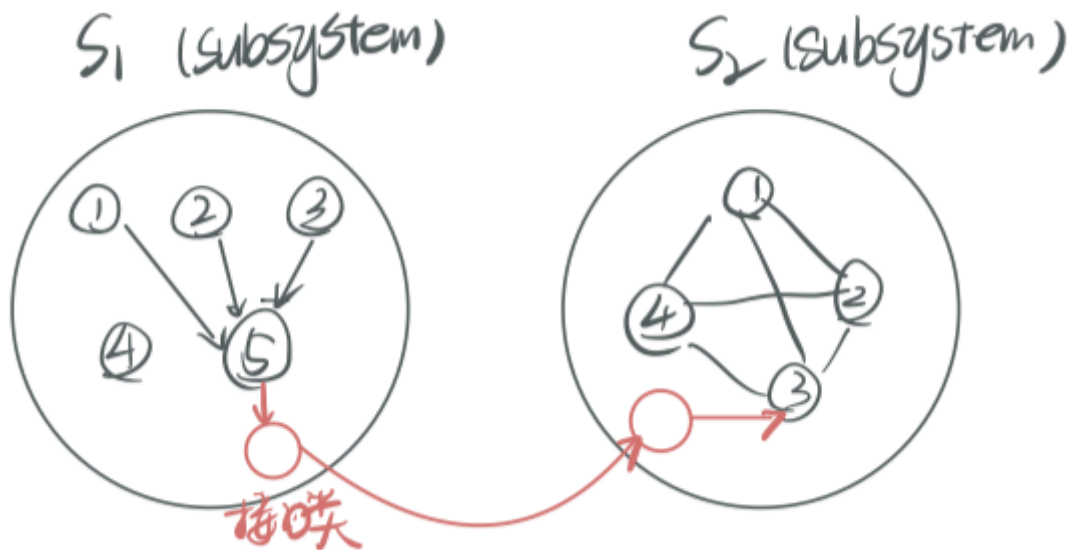


Chapter 14 详细设计Component-Level Design

软件体系架构图中各个部分（每一个类）组件的细化设计。以**分析类为依据**，对分析类进行细化补充
组件级(Component-Level)设计定义了分配给每个软件组件的数据结构，算法，接口特性和通信机制。
进行设计评审。检查设计以确定数据结构，接口，处理序列和逻辑条件是否正确。



14.1 组件

(要看)

(面向对象的角度来说) 组件不仅包含自己本身，而且还涉及一组合作类

要点：

1. Attributes——类的属性的详细定义
2. Operation——对内部方法进行定义 (Activity Diagram→对

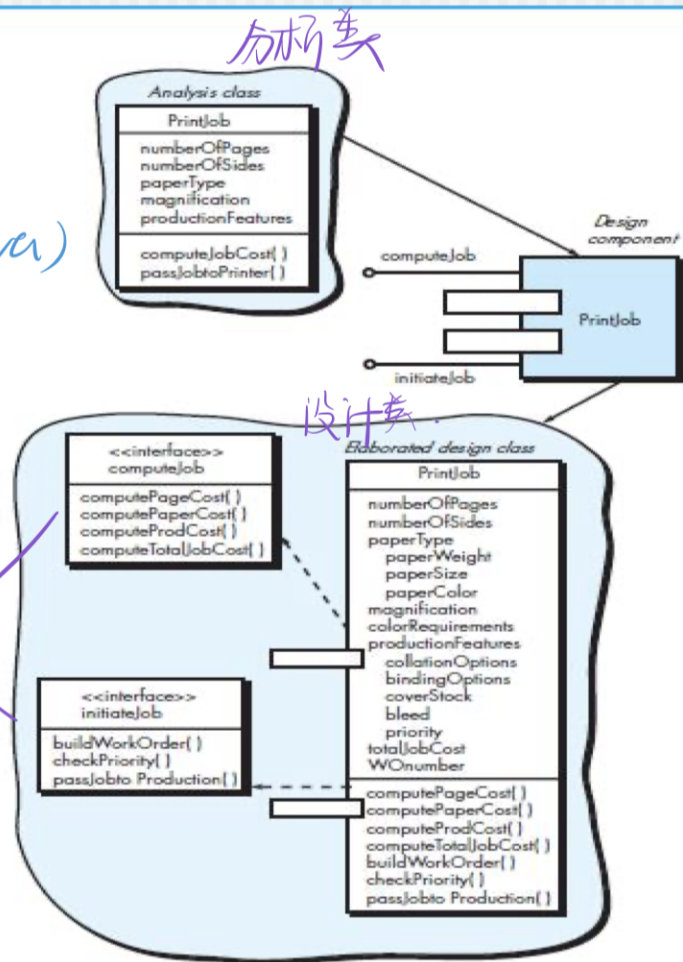
案例：打印商店

FIGURE 14.1

Elaboration
of a design
component

分析类图到设计类图
(面向对象思想 of Java)
↓
已封装的设计类

2个接口



2个接口：computeJob调用内部4个方法计算各个属性；initJob要进行建立工作，工作顺序排序等
传统的详细设计：对每个函数做设计，而不是对类做设计（不是面向对象的）

FIGURE 14.2

Structure chart for a traditional system

C语言实现 class : printJob
上面是main主程序
下面是子功能
下面-Print每一个步骤基础
与面向对象思想实现的类中
operation 相对应。

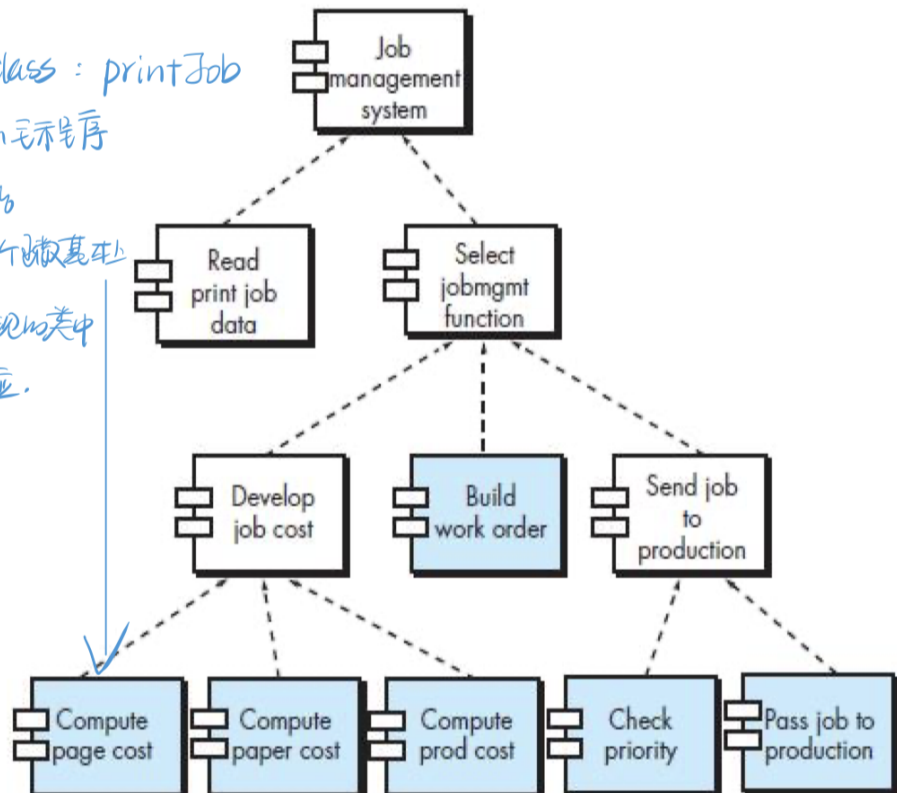
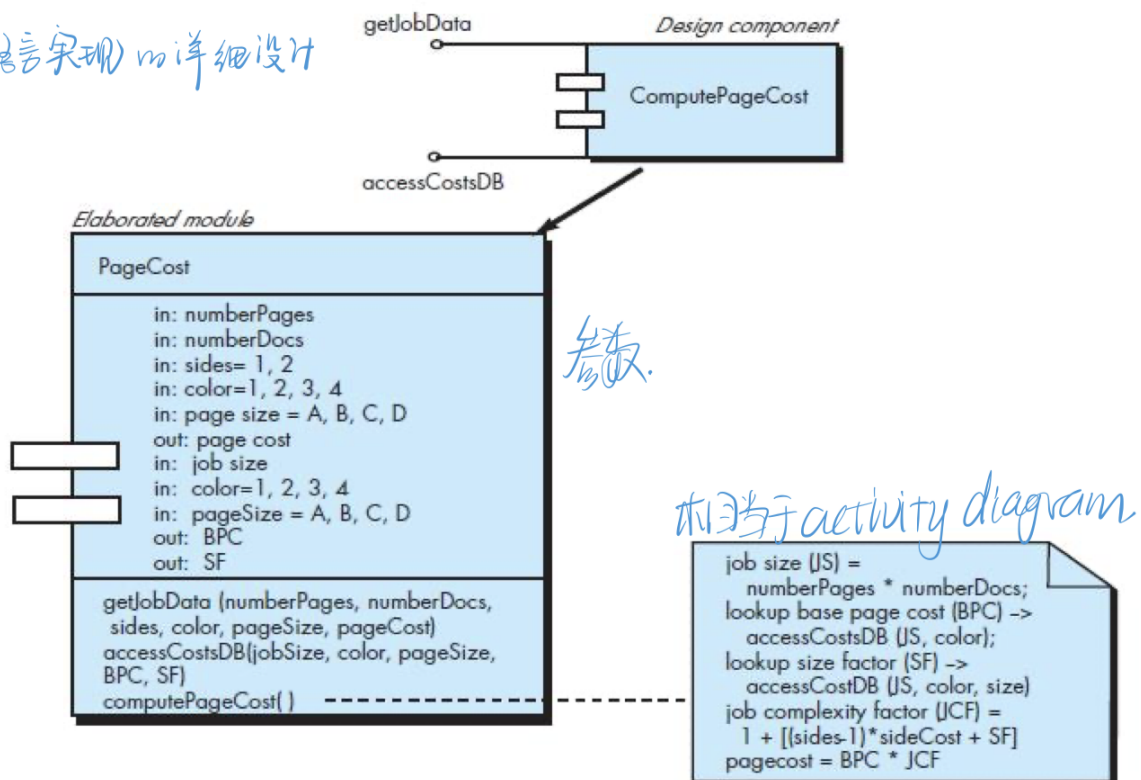


FIGURE 14.3 Component-level design for ComputePageCost

C语言实现的详细设计



14.3 进行组件级别设计

(重点重点！这章最重要的部分)

你必须将需求requirements和架构模型architectural model的信息转化为设计表示，以提供足够的细节来指导构建 (coding和testing) 活动。

详细设计步骤：

1. P认定与问题域Problem Domain对应的所有设计类（根据分析类完善业务类的设计）

Identify all design classes that correspond to the problem domain.

(对于类内的每一个方法，根据activity diagram把，后面没记完)

2. I认定与基础域Infrastructure Domain对应的所有设计类(业务功能)——e.g: GUI类，操作系统有关的组件，数据管理有关的类(对数据库访问，比如软件架构图中连接数据库组件和其他子系统的数据库连接子系统的组件)

Identify all design classes that correspond to the infrastructure domain.

一些Infrastructure Domain相关的类的知识：①只有方法没有组件的类——接口类(没有内部结构，也没有associations)；②只有属性没有方法的类——全局变量初始化类，只定义常量的类；③与业务无关的类——界面类

3. 详细阐述未获得作为可重用组件的所有设计类（确定该类和哪些类有协作，确定细节，对不用复用的业务类进行细化）——包括该类所需的所有接口，属性，操作，还得启发式设计（高内聚 低耦合）

Elaborate all design classes that are not acquired as reusable components

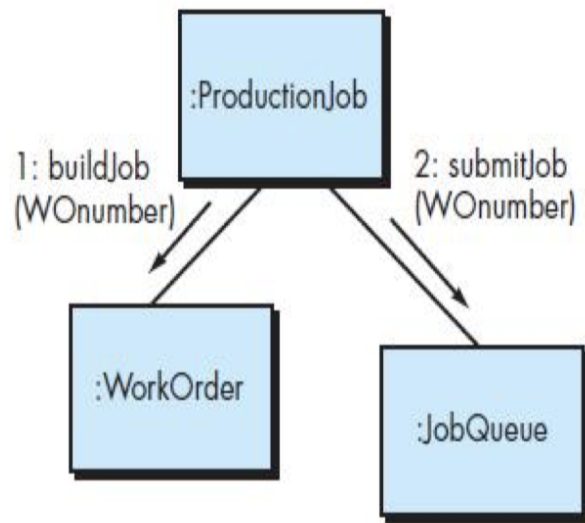
- 3.a 在类或组件协作时，指定消息详细信息(message details)

Specify message details when classes or components collaborate

**[guard condition] sequence expression (return value) :=
message name (argument list)**

FIGURE 14.6

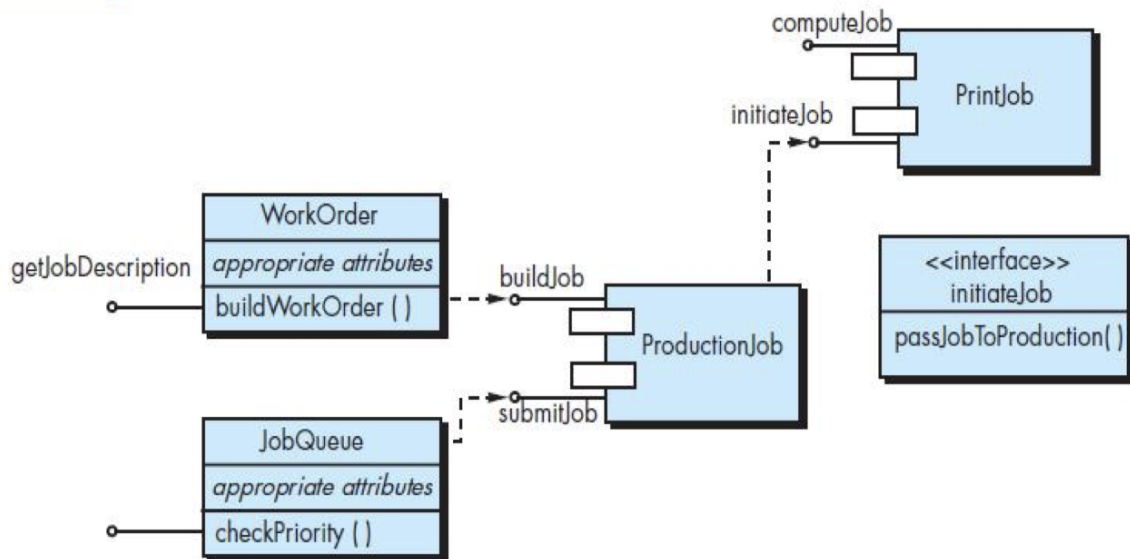
Collaboration
diagram with
messaging



3.b 定义每个**组件的接口**(先定义消息message, 再定义接口interface, 根据细节适当为每个组件提供接口)

Identify appropriate interfaces for each component

FIGURE 14.7 Refactoring interfaces and class definitions for PrintJob



3.c 定义属性, 数据的类型, 和需要的数据结构 (看属性是否合理, 需不需要为其他类的调用定义相关属性)

Elaborate attributes and define data types and data structures required to implement them.

name : type-expression = initial-value {property string}

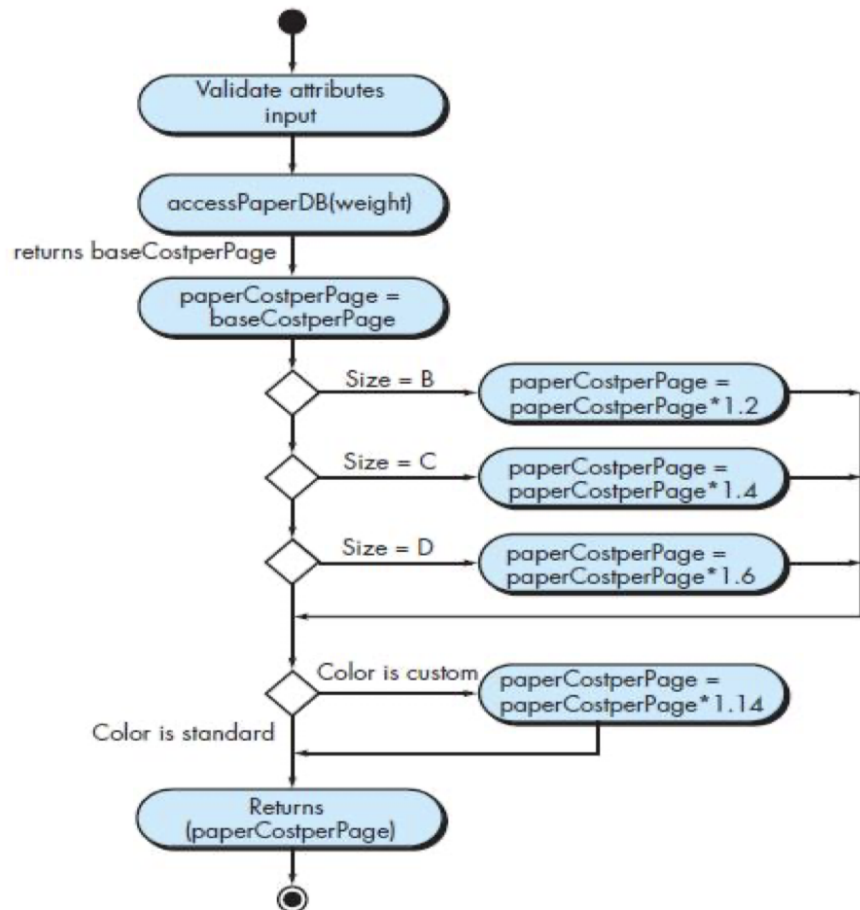
paperType-weight: string = "A" { contains 1 of 4 values 2 A, B, C, or D }

3.d 详细描述每个类每个方法的处理流程 (对每个方法要借助activity diagram/伪代码, 抽象出每个方法的功能流, 利用UML Activity Diagram或伪代码)

Describe processing flow within each operation in detail.

FIGURE 14.8

UML activity diagram for `compute-PaperCost()`



4.描述持久层的**数据源**（数据库或文件）并定义**数据管理类**(是在概要设计中进行的)——对概要设计中的数据进行review (持久的数据和相关的类) 重点检查与数据库相关类（persistent data sources）(databases and files)是否有定义

Describe persistent data sources (databases and files) and identify the classes required to manage them

5.**Review**一下每个类的状态图是否合理，对设计类进行**行为建模**

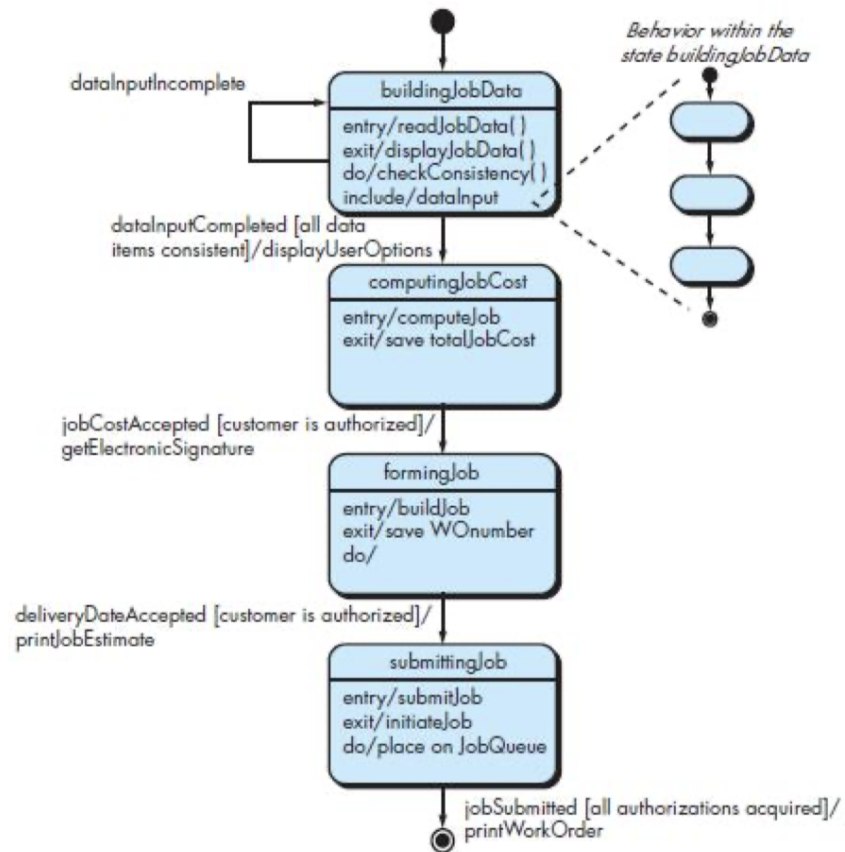
细化状态图（定义 的依据和流程图（定义接口的依据，是否有遗漏的属性或方法

Develop and elaborate behavioral representations for a class or component

Event-name (parameter-list) [guard-condition] / action expression

FIGURE 14.9

Statechart
fragment for
PrintJob class



6.画部署图，提供额外的实现细节(基于架构图)

Elaborate deployment diagrams to provide additional implementation detail.

7. 重构每个组件级设计表示(component-level design representation)，考虑备选方案。进行Review

Refactor every component-level design representation and always consider alternatives.

小结:

概要设计 → 以需求规约 (use case图)
需求分析规约中 { 功能建模
为依据 { 数据建模
行为建模

详细设计 → 对每个类进行设计

↓
包括: 业务类的细化
既有属性没有方法的类的定义
既有方法没有属性的类的定义

要以需求规约, 需求分析规约, 概要设计规约为依据
同时要考虑类之间的 cooperation.

重要: 不同系统, 不同 package, 前端后端之间的接口定义

微服务: 可独立部署, 单独测试, 独立使用 → 一个接口一个可独立工作.
不同的组件可构成一个微服务

有 系统的概念, 微服务的概念, package的概念